

人脸身份证比对查询服务（rlbd1）• API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。

版本：rlbd1 | 通信：HTTP + JSON | 编码：UTF-8

本服务提供「姓名 + 身份证号 + 人像照片」的一致性比对能力：调用方提交待核验的姓名、身份证号与一张人像照片，服务返回比对结论（含比对分值与结论码）。请求信封统一为

`appKey / sign / encryptionType / body` + MD5 加签，响应统一为 `head / body` 两段式；比对结果为一个结构化对象，服务将其序列化为 JSON 字符串置于 `body.result.range`，由调用方自行解析。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「人脸身份证比对查询服务（rlbd1）」的第三方产品技术开发人员与日常运维人员。

1.2 接入须知

1. 正式服务地址（任选其一，两域名等价）：

- `http://aiszcloud.cn:8080`

- `http://aiszcloud.com.cn:8080`

1. 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。

2. 照片要求：格式为 JPG 或 PNG；宽、高均大于 15px 且小于 4000px；文件大小不超过 500KB；须五官清晰可见、正脸、不旋转、无刘海遮挡、背光不过强。

1.3 接口说明

项目	说明
请求方式	POST（成功查得数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（建议客户端读超时 ≥ 10 秒；照片上传占用传输时间，请预留充足超时）
HTTP 状态码	恒为 200；业务结果与错误均通过响应体的 head.errorCode / body.code 表达
签名	调用时需对 body 中的业务参数 + 我方分配的 appSecret 进行 MD5 加签，详见「二、鉴权与加签」

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，**按查得成功条数计费**（见「五、计费说明」）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y89r1bd1	String	是	我方分配给客户的公开标识
sign	0528999dd55c025b8f36fc72dceb1f63	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

说明: `appKey` / `sign` / `encryptionType` **不参与**签名计算。调用方传入的 `body` 字段 (姓名/身份证号/照片) 一律为**明文**。

2.2 鉴权校验顺序

网关按以下顺序校验, 任一失败立即返回对应 `head.errorCode` (不进行比对、不计费):

1. `appKey` 是否存在 → 否则 505001
2. `appKey` 是否匹配到账户 → 否则 505004
3. 账户是否有效 (启用且在有效期内) → 否则 505007
4. 签名是否正确 → 否则 505002

2.3 加签方式

1. 取出 `body` 中**所有非空的业务参数** (不含文件/字节流类型, 不含值为空的参数)。
2. 按参数名 (key) 的 **ASCII 升序**排序; 首字符相同则依次比较后续字符。
3. 将排序后的参数按 `参数名 参数值` 直接拼接, 最后追加 `appSecret`, 得到**待签名串**。
4. 对待签名串做 **MD5**, 取 **32 位小写十六进制**字符串, 赋值给 `sign`。

示例: `body = { "name": "张三", "idCard": "330129199109094312", "url": "http://img.example.com/a.jpg" }`, `appSecret = "<你的密钥>"`。

按 ASCII 升序排序后 key 顺序为 `idCard` → `name` → `url`, 待签名串为:

```
idCard330129199109094312name张三urlhttp://img.example.com/a.jpg<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示: 拼接顺序由 key 的 ASCII 决定, 请勿写死字段顺序; 新增字段时排序会自动变化。若以 `image` (Base64) 方式传图, `image` 亦作为普通业务参数参与加签。

2.4 加签代码示例

Java

```

import java.security.MessageDigest;
import java.util.Map;
import java.util.TreeMap;

public static String sign(Map<String, String> body, String appSecret) throws Exception {
    TreeMap<String, String> sorted = new TreeMap<>();
    for (Map.Entry<String, String> e : body.entrySet()) {
        if (e.getValue() != null && !e.getValue().isEmpty()) sorted.put(e.getKey(), e.getValue());
    }
    StringBuilder sb = new StringBuilder();
    for (Map.Entry<String, String> e : sorted.entrySet()) sb.append(e.getKey()).append(e.getValue());
    sb.append(appSecret);
    byte[] d = MessageDigest.getInstance("MD5").digest(sb.toString().getBytes("UTF-8"));
    StringBuilder hex = new StringBuilder();
    for (byte b : d) hex.append(String.format("%02x", b));
    return hex.toString();
}

```

Python

```

import hashlib

def sign(body: dict, app_secret: str) -> str:
    parts = [f"{k}{v}" for k, v in sorted(body.items()) if v]
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest()

```

Go

```
import (
    "crypto/md5"
    "encoding/hex"
    "sort"
    "strings"
)

func Sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" {
            keys = append(keys, k)
        }
    }
    sort.Strings(keys)
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:])
}
```

三、接口列表

3.1 人脸身份证比对 (rlbd1)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxRLBD1
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/querySrmxRLBD1 (或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/querySrmxRLBD1)
鉴权	appKey + MD5 签名 (见第二章)

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
name	张三	String	是	姓名，明文传入
idCard	330129199109094312	String	是	身份证号（18 位，末位可为 X ），明文传入
image	/9j/4AAQSkZ...	String	否	人像照片的 Base64 字符串。image 与 url 二选一
url	http://img.example.com/a.jpg	String	否	人像照片的可访问 URL。image 与 url 二选一

name、idCard 必填；image 与 url 至少提供一个（同时提供时以 image 为准）。参数缺失或格式非法（身份证号不符、照片缺失）将返回 head.errorCode = 505062，不进行比对、不计费。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y89rlbd1",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "body": {
    "name": "张三",
    "idCard": "330129199109094312",
    "url": "http://img.example.com/a.jpg"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	356	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 查得比对结论【计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	a1b2c3d4	String	交易流水号，与 head.logId 同值，对账时提供其一即可
result	{...}	Object	业务内容，仅 code = 001 时存在
result.range	" {\"score\":932.26,...}"	String	比对结果的 JSON 字符串 ，调用方需 JSON.parse 后使用，结构见 3.1.5

3.1.4 响应示例

① 查得比对结论（计费）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 356, "timestamp": 1718456789012 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "1kf9x2ab",
    "uid": "a1b2c3d4",
    "result": {
      "range": "{\"score\":932.26,\"msg\": \"系统判断为同一人\", \"incorrect\":100, \"sex\": \"男\"}"
    }
  }
}
```

② 网关级错误（无 body）

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d6", "time":
}
```

3.1.5 result.range 结构说明

result.range 是比对结果对象序列化后的 JSON 字符串。调用方应先对该字符串做一次 JSON.parse / 反序列化，再读取以下字段：

字段	类型	说明
score	Number	比对结果分值，范围 0 – 1000 。同一人：分数 ≥ 700；疑似同一人：600 ≤ 分数 < 700；非同一人：分数 < 600
msg	String	比对结果的文字描述
incorrect	int	比对结果码，取值见下表
sex	String	性别
birthday	String	出生日期（ yyyyMMdd ）
address	String	籍贯

incorrect 比对结果码：

incorrect	含义
100	比对成功
101	身份证号码与姓名不一致
103	身份核验成功，数据非法
109	身份核验成功，库中无照片
110	身份核验成功，特征提取失败
111	身份核验成功，检测到多于一张人脸
112	身份核验成功，图片不合法

说明：result.range 只承载比对业务数据，不含任何内部处理标识；对账请使用 head.logId / body.uid 。具体字段以本服务实际返回为准，建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因字段增减导致解析失败。

解析示例（JavaScript）：


```
const range = JSON.parse(resp.body.result.range);
if (range.incorrect === 100 && range.score >= 700) {
  console.log("判定为同一人，分值:", range.score);
}
```

3.2 成功查得数查询（扩展接口）

查询本账户累计成功查得比对结论的次数，用于客户侧自助监控。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaRLBD1
完整地址	http://aiszcloud.cn:8080/v1/openapi/zlx/quotaRLBD1 （或 http://aiszcloud.com.cn:8080/v1/openapi/zlx/quotaRLBD1 ）
鉴权	与主接口一致（请求体中携带 appKey + sign 信封；body 可为 {}，此时 sign = MD5(appSecret) ）

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1536
}
```

参数	说明
status	账户状态（ACTIVE/SUSPENDED 等）
serviceUsed	累计成功查得比对结论的次数（仅统计查得成功 code=001 ）
totalCalls	累计调用次数

说明：无任何额度上限拦截，仅做成功查得数统计。

3.3 健康检查

项目	内容
路径	GET /healthz
完整地址	http://aiszcloud.cn:8080/healthz (或 http://aiszcloud.com.cn:8080/healthz)
鉴权	无
响应	HTTP 200, 纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功 (业务结果见 body.code)
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505062	数据请求异常	参数非法 / 照片不合格 / 服务处理异常 / 系统错误 (默认错误码)

参数非法、照片质量不合格、图片过大、比对服务异常等无法产出可计费结论的情况，统一归一为网关 505062，**不计费**，并经异步复查兜底。

4.2 业务结果码 body.code (仅 errorCode = 0 时)

code	含义	是否计费
001	查得比对结论	计费

说明：本服务仅在成功产出可计费的比对结论时返回 001 并计费；其余情况通过 head.errorCode (505062 等) 表达，不产生 body 。

五、计费说明

- 仅当返回 body.code = 001 (查得比对结论) 时，才计入成功查得数并对客户计费。
- 网关级错误 (head.errorCode 非 0: 鉴权失败、参数非法、照片不合格、服务异常等) **一律不计费**。

- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 `appKey`、`appSecret`；服务地址见「1.2 接入须知」。
2. 按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过成功查得数查询接口（3.2）监控调用量。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（ ≥ 10 秒，含照片上传传输时间）。
- 收到网络超时/无响应时**可安全重试**：网关基于内部流水号做幂等，不会因重试重复计费。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 505062、鉴权错误 505001/505002/505004），应先修正再发起。

6.3 错误处理建议

现象	排查方向
505001 / 505004	检查 <code>appKey</code> 是否正确、是否用错环境账户
505002	检查签名算法（排序/空值剔除/UTF-8/小写 hex）
505007	联系商务确认账户状态与有效期
505062	检查 <code>name</code> / <code>idCard</code> 是否齐全合法、 <code>image</code> / <code>url</code> 是否至少提供一个、照片是否符合 1.2 的规格；若入参正常仍持续出现，记录 <code>logId</code> 联系我方

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 服务地址（`aiszcloud.cn:8080` 或 `aiszcloud.com.cn:8080`）、`appKey`、`appSecret`、环境匹配无误
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ `name` / `idCard` 均以**明文**传入，`image` / `url` 二选一提供
- ☐ 照片符合 1.2 规格（格式/尺寸/大小/正脸清晰）
- ☐ 能正确解析 `head.errorCode` 与 `body.code` 两级状态
- ☐ 已实现对 `result.range` 字符串的二次 `JSON.parse` 解析与字段容错
- ☐ 已实现超时重试（依赖幂等，不重复计费）

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign ，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId) ，排障/对账唯一凭据
uid	交易流水号 (= body.uid ，与 head.logId 同值) ，对账用
range	比对结果的 JSON 字符串，需二次解析（结构见 3.1.5）